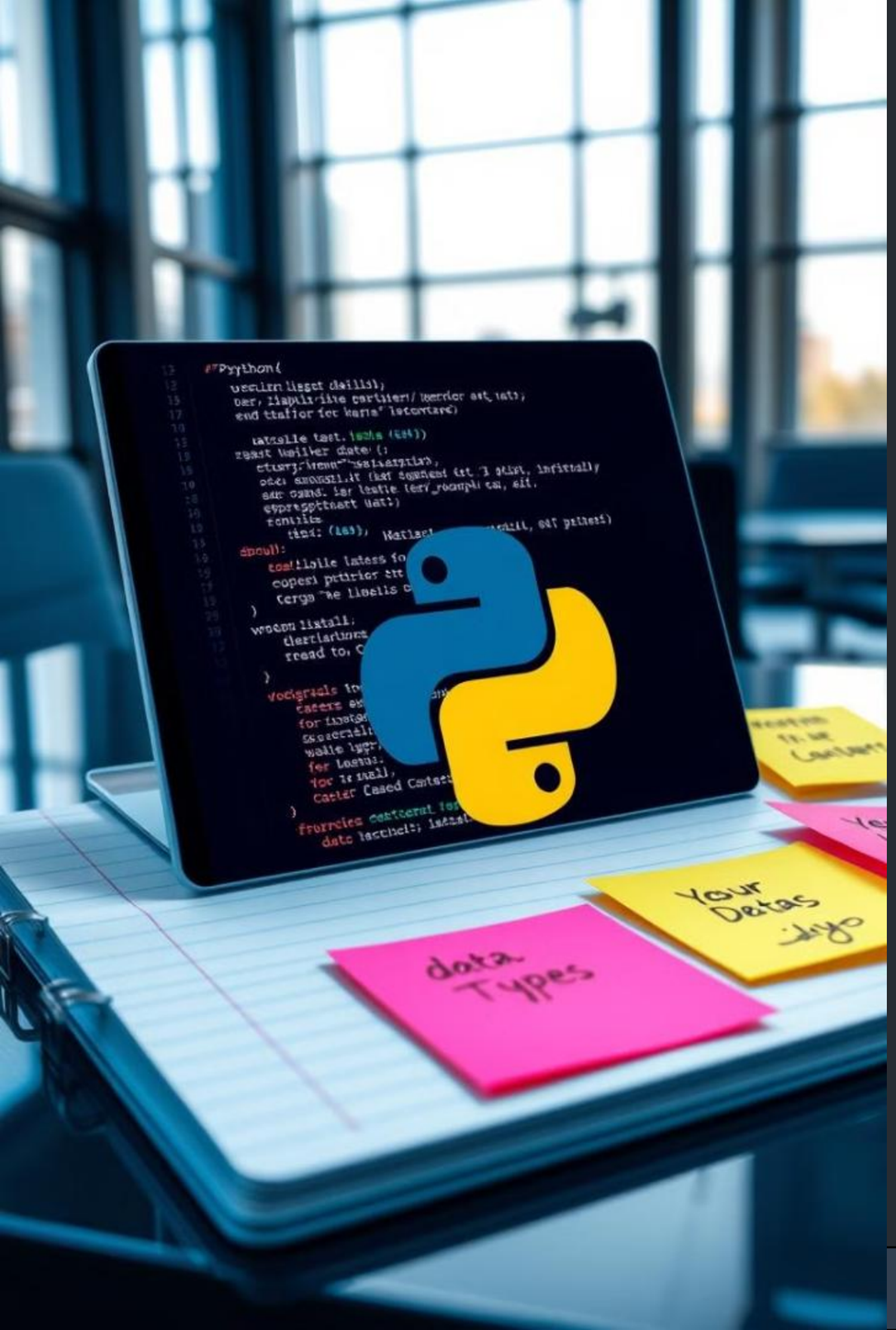


Условные операторы

СЦ

Цехош София Ивановна

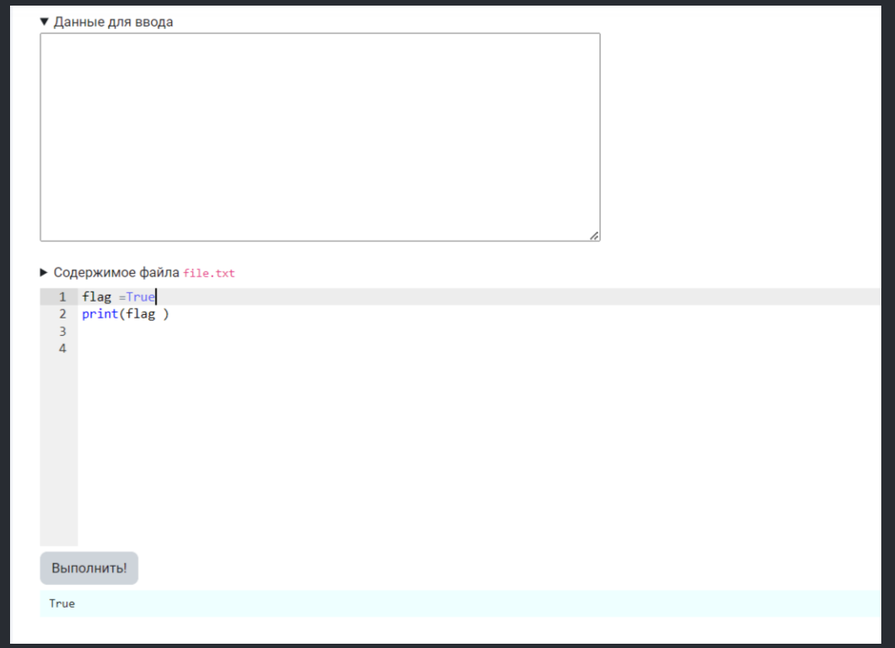
Дата обращения 01.10.2026

Булев тип (boolean)

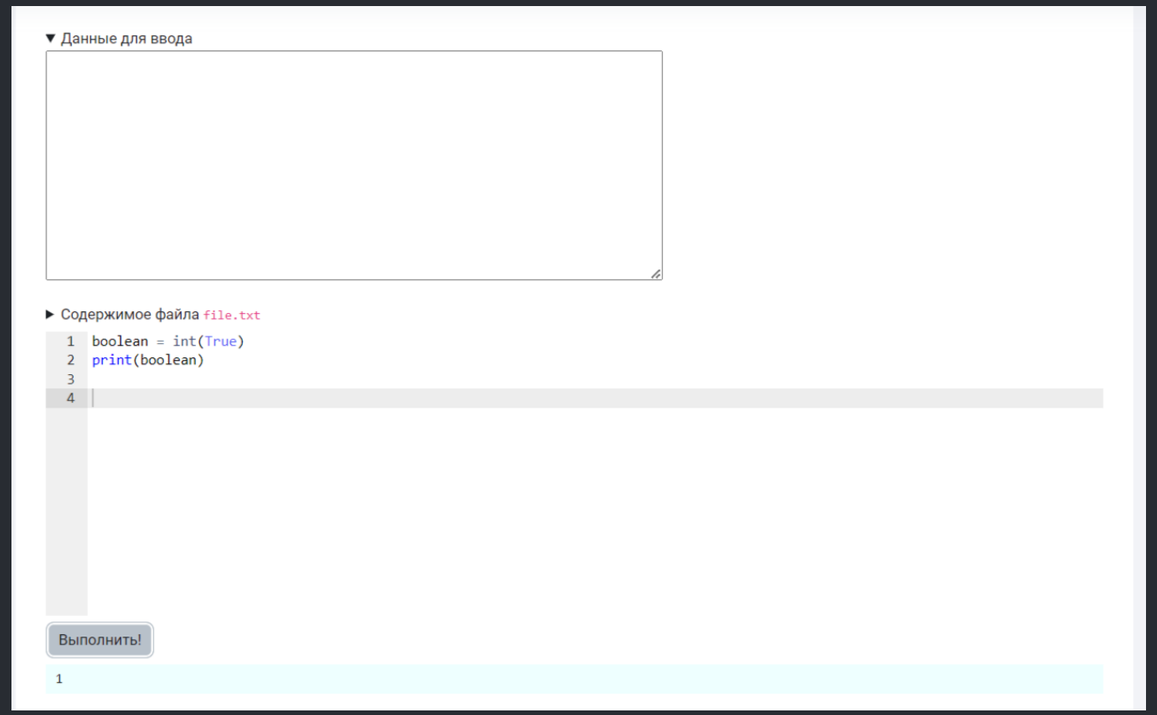
1. True и False

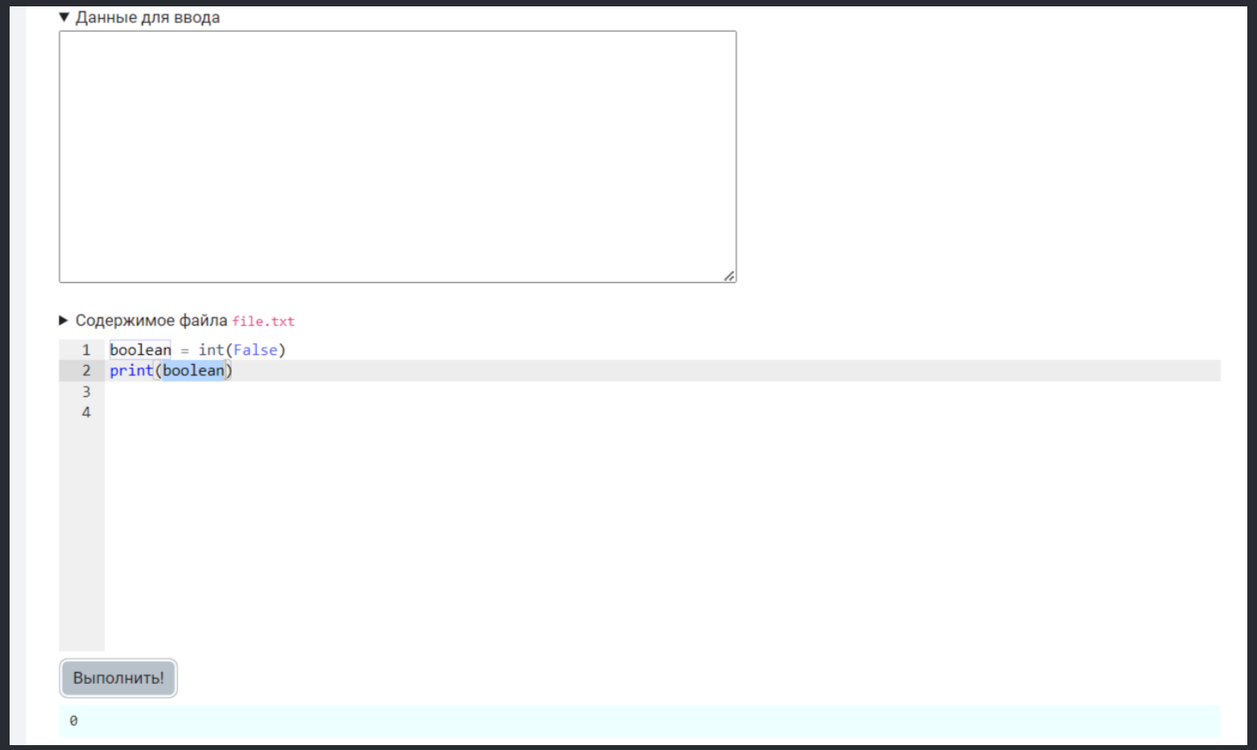
- Булев тип (boolean) в Python представляют два логических значения: True (истина) и False (ложь). Они используются для хранения и обработки логических условий.

- Логические значения True и False не заключаются в кавычки и всегда начинаются с большой буквы T и F.



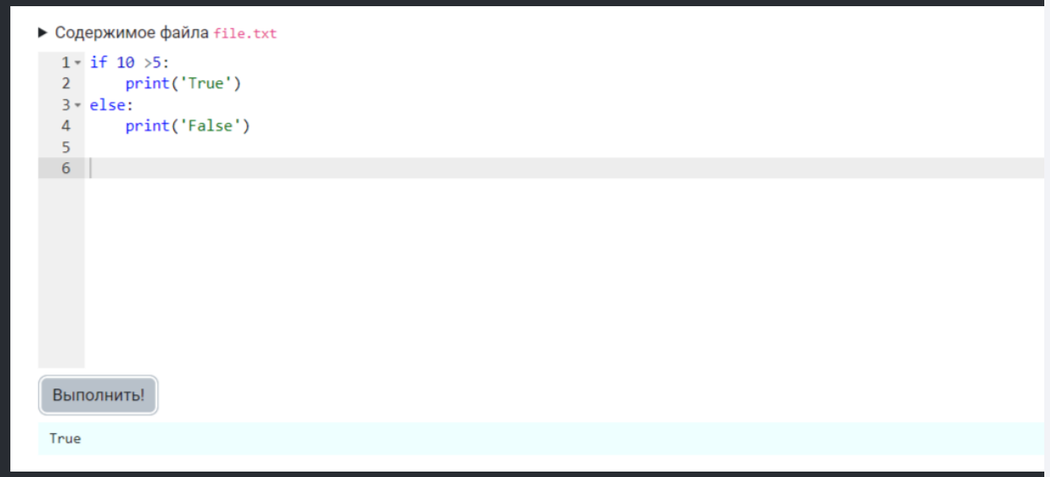
Булев тип (boolean)

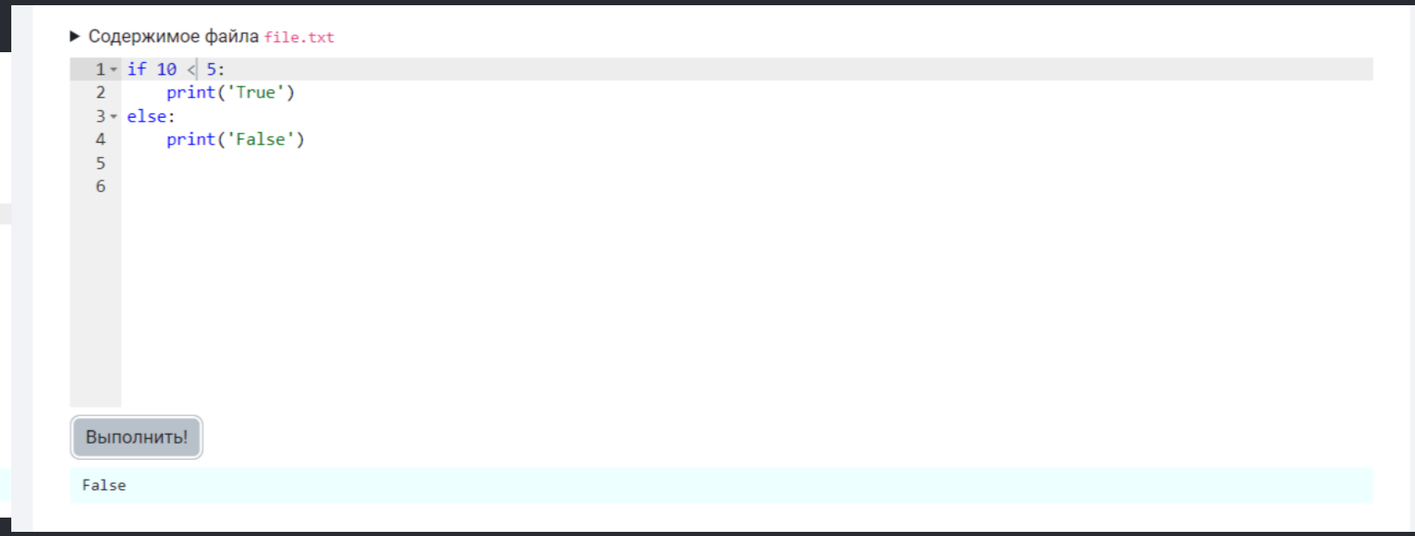




- Хранение переменной с типом boolean в памяти компьютера 1 (True) или 0 (False).

Операторы сравнения





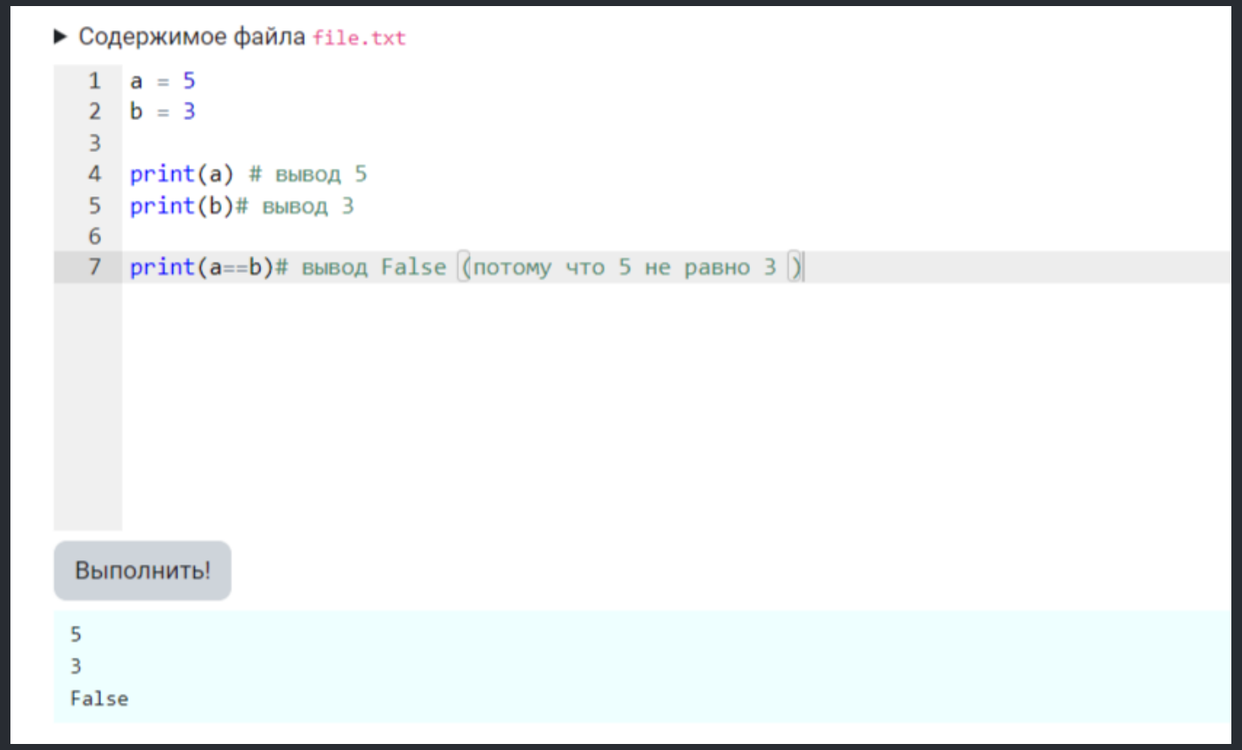
Операторы сравнения могут возвращать всего два результата: True и False.

Оператор	Название
==	Равно
!=	Не равно
<	Меньше чем
>	Больше чем
<=	Меньше или равно
>=	Больше или равно

Операторы сравнения

= при создание переменных (инициализации).

== сравнение (переменных, выражений)



Оператор	Название
==	Равно
!=	Не равно
<	Меньше чем
>	Больше чем
<=	Меньше или равно
>=	Больше или равно

- Оператор равенства == возвращает True, если числа равны, и False в противном случае.

```
a = 12
b = 12
print(a == b)
# Вывод
>> True
```

```
a = 8
b = 7
print(a == b)
# Вывод
>> False
```

- Оператор больше > возвращает True, если первое число больше второго, и False в противном случае.

```
a = 8
b = 7
print(a > b)
print(b > a)
# Вывод
>> True # a > b
>> False # b > a
```

- Оператор больше или равно >= возвращает True, если первое число больше второго или равно ему, и False в противном случае.
- Оператор меньше или равно <= возвращает True, если первое число меньше второго или равно ему, и False в противном случае.

```
c = 200
d = 200
print(c >= d)
print(d <= c)
# Вывод
>> True
>>  True
```

- Оператор неравенства != возвращает True, если числа не равны, и False в противном случае.

```
a = 8
b = 7
print(a != b)
# Вывод
>> True
```

Булев тип (boolean)

2. Логические операторы:

- **Оператор and (И) -** логическое умножение - конъюнкция. Конъюкция **истинна** в том случае, когда **все условия истинны**.

```
print(10 > 0 and 5 > 0)# Первое условие  True, второе True. Итог True.
print(10 % 2 == 0 and 12 % 2 == 0) # оба числа четные
# Вывод:
>> True
>> True
```

Первое логическое значение	Второе логическое значение	Результат (Первое and Второе)
True	True	True
False	True	False
True	False	False
False	False	False

Если хотя бы **один** из операторов **равен False**, то результат оператора будет **равен False**.

```
print(10 > 100 and 5 > 0 and 10 > 0)  # Первое условие False, второе True, а третье True.  Итог False.
# Вывод:
>> False
```

Результатом этих операций также будет True или False в зависимости от исходных. значений.

Булев тип (boolean)

Логические операторы:

- **Оператор or (ИЛИ) -** логическое сложение (дизъюнкция). Оператор **or** требует выполнения **ХОТЯ БЫ одного условия**.

```
print(10 > 100 or 5 > 100 or 10 > 0) # Первое условие False, второе False, а третье True. Итог True!
print(1 == 0 or 2 == 0) # Первое условие False, второе False. Итог False(оба условия ложны).
```

```
# Вывод:
>> True # Вывод первой строки
>> False # Вывод второй строки
```

Первое логическое значение	Второе логическое значение	Результат (Первое or Второе)
True	True	True
False	True	True
True	False	True
False	False	False

Результатом этих операций также будет True или False в зависимости от исходных.

Булев тип (boolean)

Логические операторы:

- **Оператор not (НЕ) -** логическое отрицание (инверсии). Оператор **not изменяет** значение **на противоположное**.

```
print(not False) # противоположное False является True
print(not True) # противоположное True является False
print(not 2 == 0) # два не равно 0 (является истиной)

# Вывод:
>> True
>> False
>> True
```

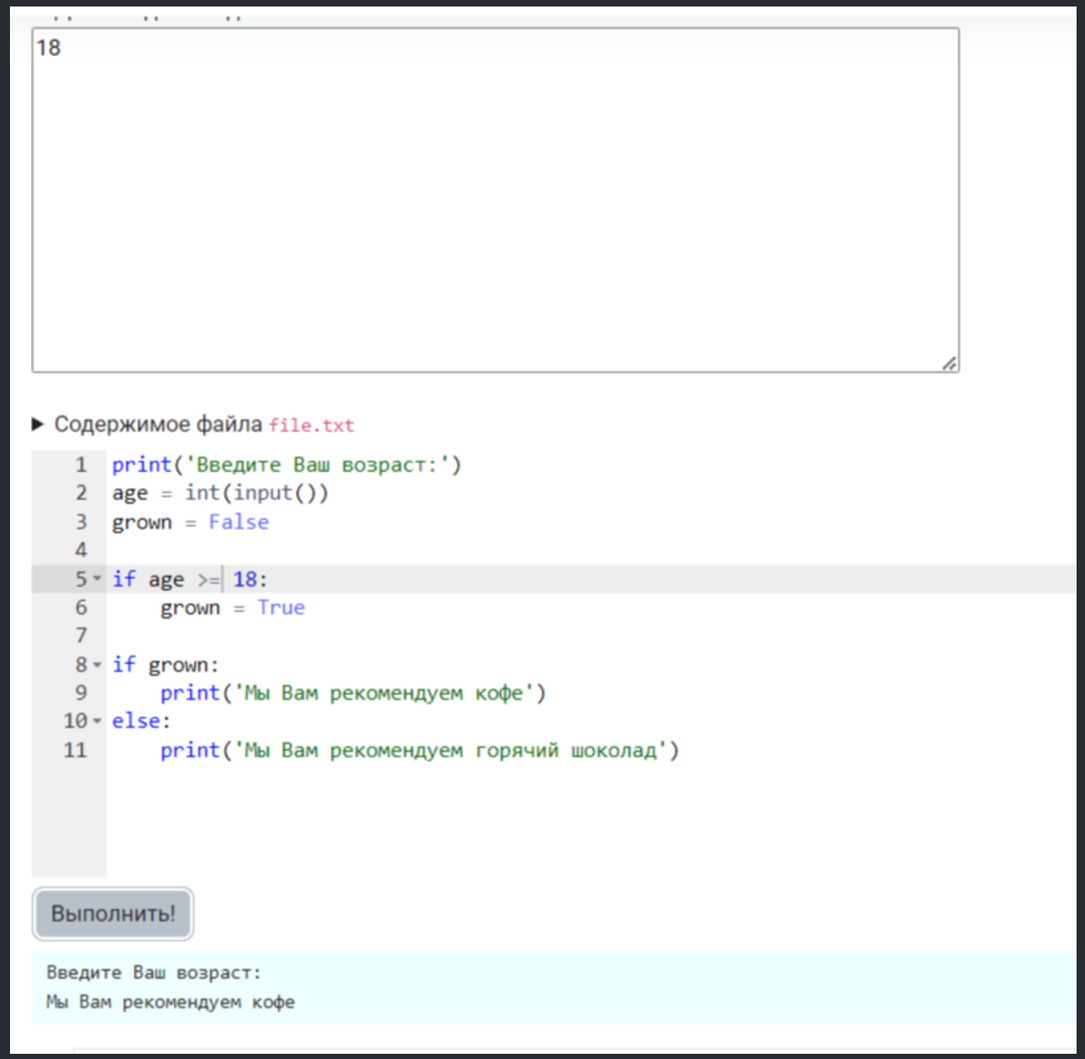
Оператор not выполняется в приоритете.

```
print(not 1 == 0 or 2 == 0) # True, значение первого условия  инвертировано
# Вывод:
>> True
```

Логическое значение	Результат (not логическое значение)
True	False
False	True

Результатом этих операций также будет True или False в зависимости от исходных.

Булевы типы (boolean)



Результат выполнения конструкции **age >= 18** будет либо True либо False.



Укороченный вариант кода.

Булевы типы (boolean)



Другие типы данных, такие как числа и строки, могут быть приведены к булевым значениям. Это позволяет использовать их в логических выражениях и условиях.

Булев тип (boolean)

```
print(bool(1)) # True
print(bool(0)) #  False
print(bool(10.2)) #  True
print(bool('')) #  False
print(bool('Мир')) #  True
```

В условии любой тип данных неявно приводится к bool. Пустая строка и ноль равны False.

If, elif, else

3. Условные операторы

if (если) – основное условие. **if** – может работать **без** elif и else.

elif – сокращение от else(иначе) if (если). **Обязательно в конструкции elif необходимо применять if (перед elif).**

else (иначе) – используется в условных операторах для выполнения блока кода, когда исходное условие не выполняется.

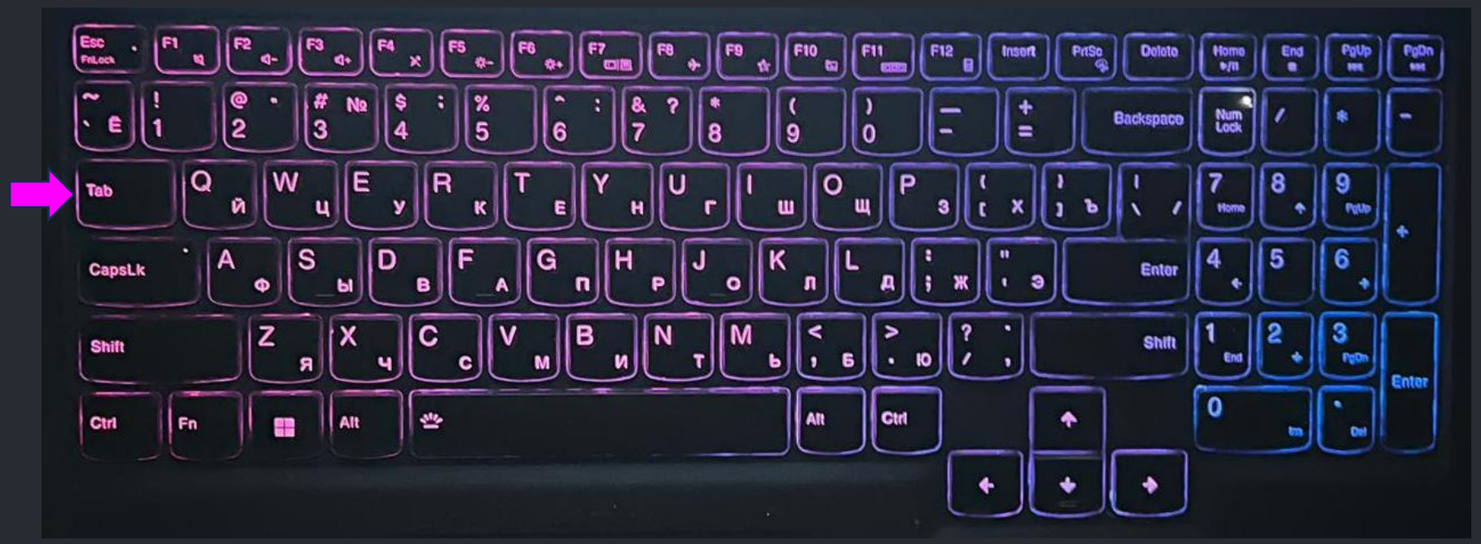
Булевы значения широко используются в условных операторах, таких как if, else и elif, которые позволяют управлять выполнением кода в зависимости от результата логической проверки.

If, elif, else

```
age = int(input("Введите ваш возраст: "))
if age < 13:
    print('Ребенок')
elif age >=13 and age < 20:
    print('Подросток')
elif age >= 20 and age <= 65:
    print('Взрослый человек')
else:
    print('Пожилой человек')
```

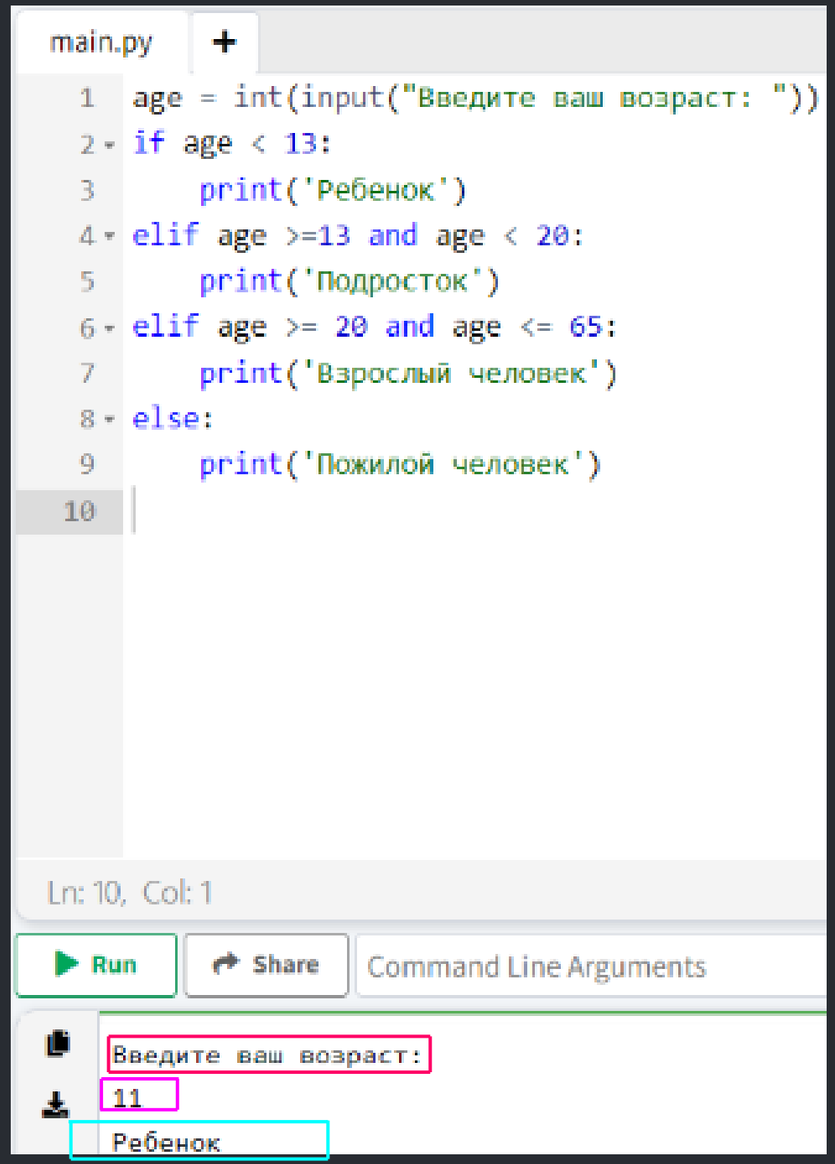
```
if  age < 13: # Обязательное двоеточие перед блоком кода
    print('Ребенок')

# Обратите внимание на отступ в четыре пробела!!!
# или одну табуляцию (кнопка TAB)
```



```
age = int(input("Введите ваш возраст: "))
if age < 13:
    print('Ребенок')
elif age >=13 and age < 20:
    print('Подросток')
elif age >= 20 and age <= 65:
    print('Взрослый человек')
else:
    print('Пожилой человек ')

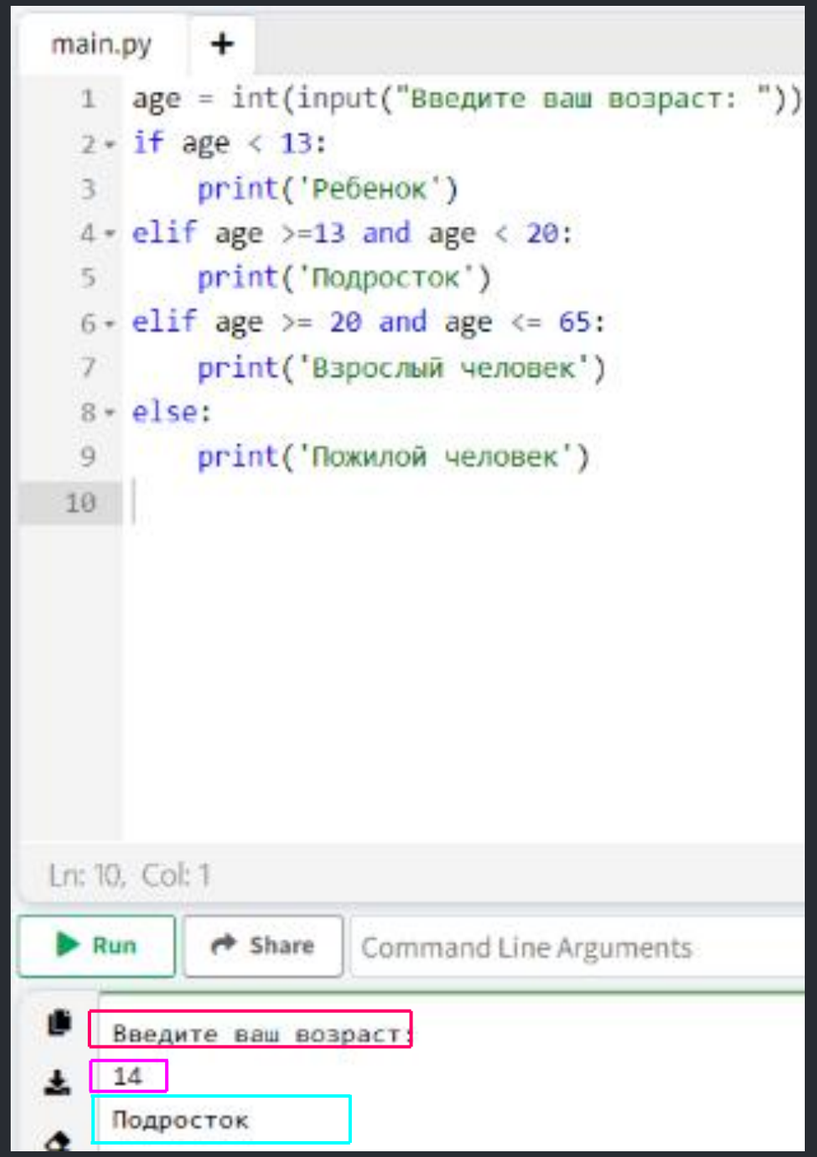

#Вывод
Введите ваш возраст:
11
Ребенок
```



Интерпретатор проверяет истинность условия, где оператор **if** истина (**в данном коде**).

```
age = int(input("Введите ваш возраст: "))
if age < 13:
    print('Ребенок')
elif age >=13 and age < 20:
    print('Подросток')
elif age >= 20 and age <= 65:
    print('Взрослый человек')
else:
    print('Пожилой человек ')

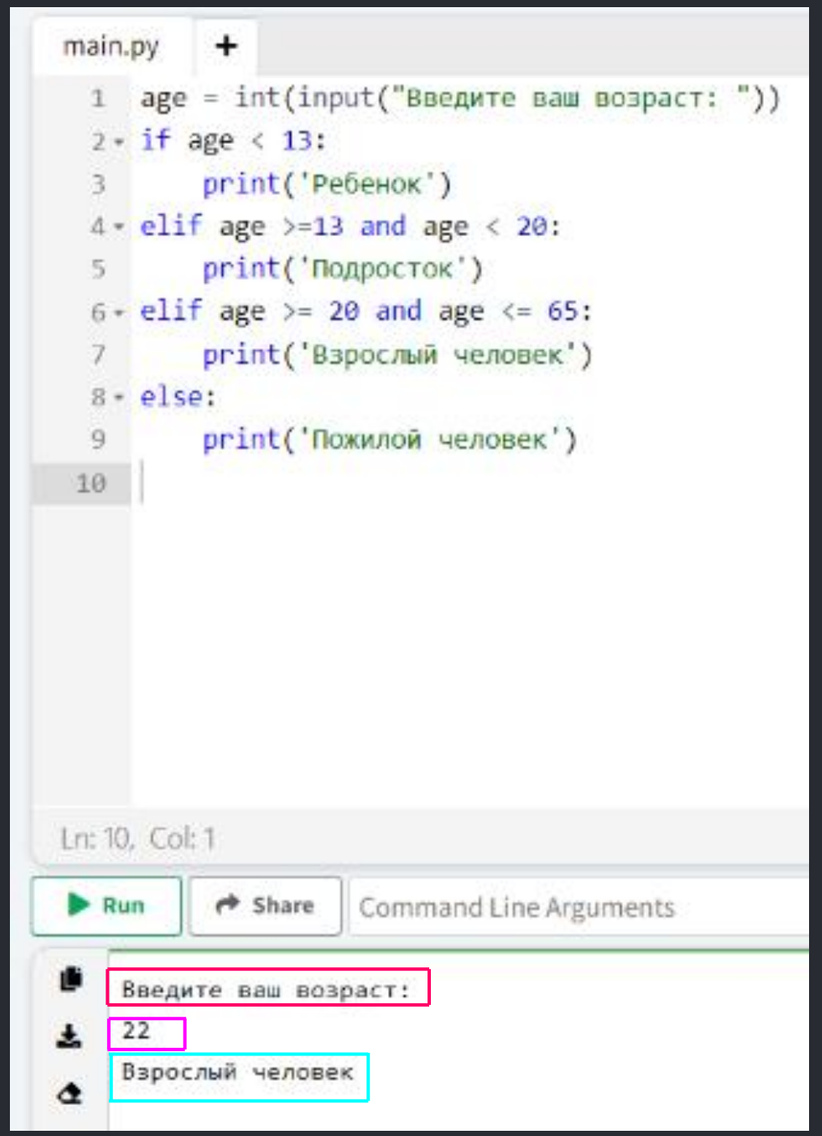

#Вывод
Введите ваш возраст:
14
Подросток
```



Если условие в операторе **if** ложно, то интерпретатор проверяет истинность условия операторов **elif**, всех по очереди.

```
age = int(input("Введите ваш возраст: "))
if age < 13:
    print('Ребенок')
elif age >=13 and age < 20:
    print('Подросток')
elif age >= 20 and age <= 65:
    print('Взрослый человек')
else:
    print('Пожилой человек ')

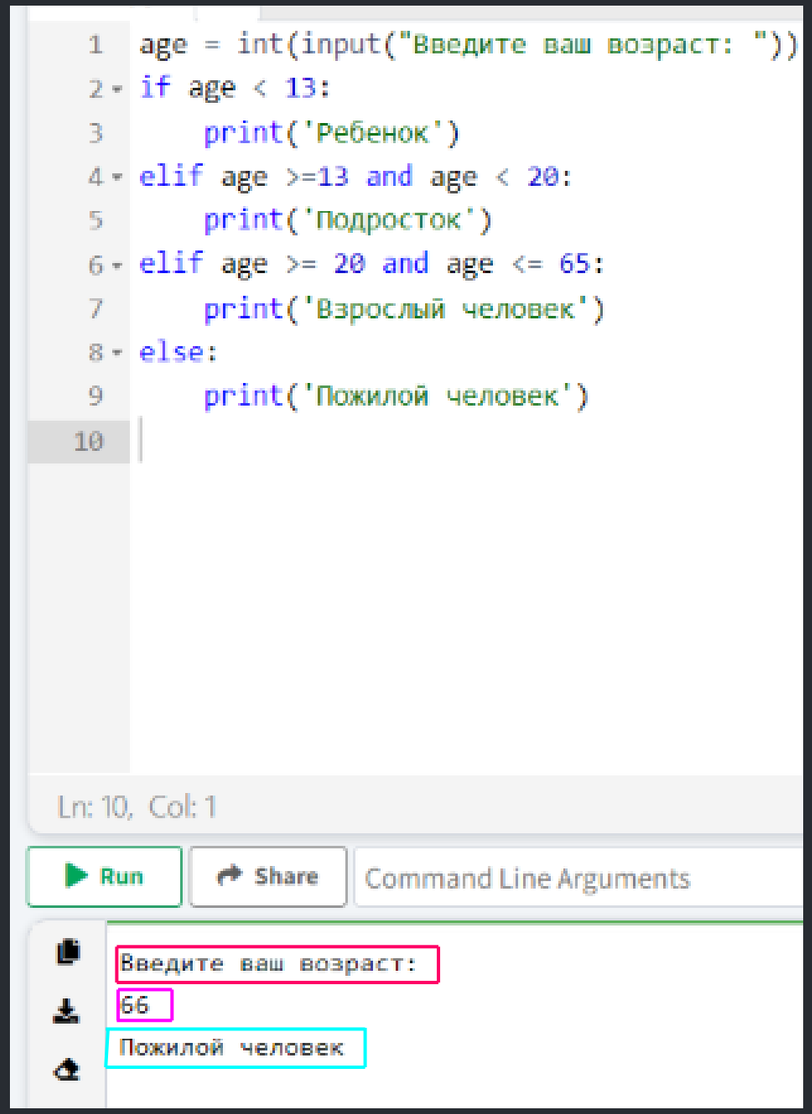

#Вывод
Введите ваш возраст:
22
Взрослый человек
```



Если условие в операторе **if** ложно, то интерпретатор проверяет истинность условия операторов **elif**, всех по очереди.

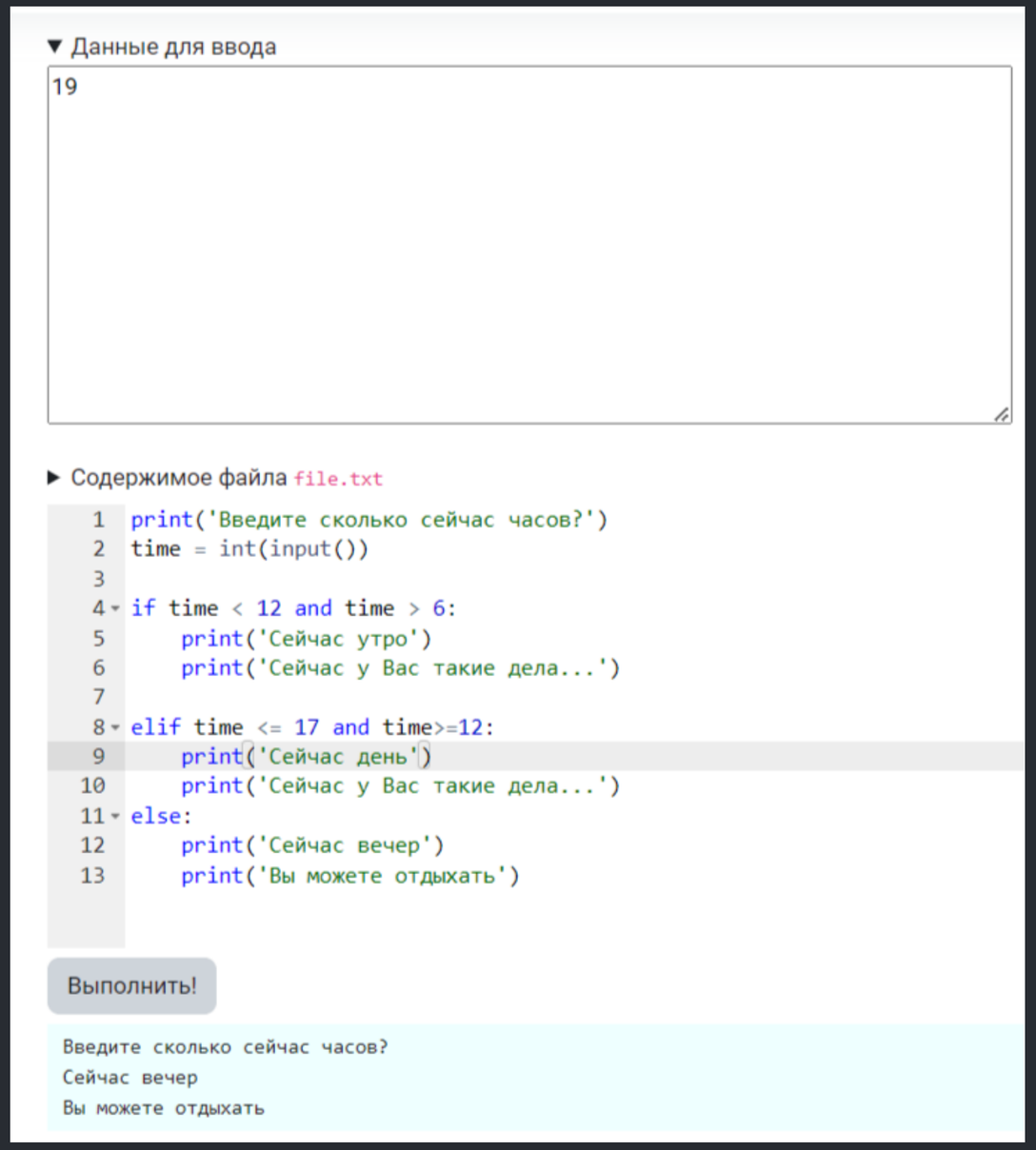
```
age = int(input("Введите ваш возраст: "))
if age < 13:
    print('Ребенок')
elif age >=13 and age < 20:
    print('Подросток')
elif age >= 20 and age <= 65:
    print('Взрослый человек')
else:
    print('Пожилой человек ')


#Вывод
Введите ваш возраст:
66
Пожилой человек
```



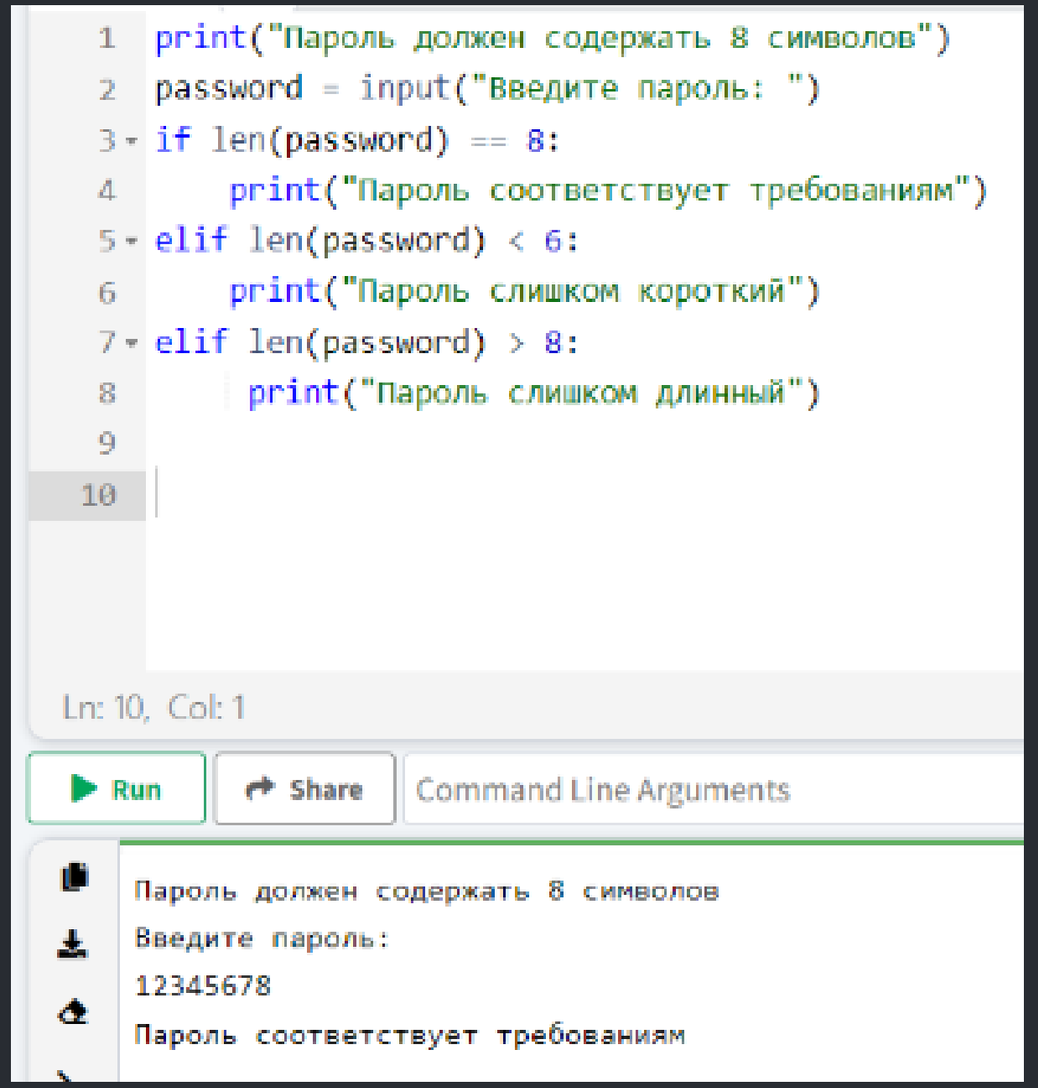
Если условие в операторе **if** ложно, то интерпретатор проверяет истинность условия операторов **elif**, всех по очереди. Если все условия ложны, то выполнится оператор **else**.

if, elif, else



Если условие в операторе **if** ложно, то интерпретатор проверяет истинность условия операторов **elif**, всех по очереди. Если все условия ложны, то выполнится оператор **else**.

```
print("Пароль должен содержать 8 символов")
password = input("Введите пароль: ")
if len(password) == 8:
    print("Пароль соответствует требованиям")
elif len(password) < 6:
    print("Пароль слишком короткий")
elif len(password) > 8:
     print("Пароль слишком длинный")
```



```
print('Пароль должен быть 8 символов и содержать  символ @ ')
password = input("Введите пароль:")
print(password)
if len(password) == 8:
    print("Пароль соответствует требованиям")
if len(password) < 6:
    print("Пароль слишком короткий")
if len(password) > 8:
    print("Пароль слишком длинный")
If  "@" not in password:
    print('пароль должен содержать символ @')
```

